TECHNICAL REPORT

1. ABSTRACT

This report details the development of a deep learning solution for binary change detection in Earth Observation (EO) and Synthetic Aperture Radar (SAR) imagery. The primary objective was to classify image pairs into 'No-Change' or 'Change' categories. Our approach utilized a modified Siamese Convolutional Neural Network (CNN) architecture designed to process 3-channel grayscale inputs (224x224 pixels). A weighted Cross-Entropy Loss function was employed to mitigate the effects of a 2:1 class imbalance, and the model was trained for 5 epochs using an Adam optimizer. Despite achieving perfect scores (IoU, Precision, Recall, F1 all 1.00) on both validation and test sets, which suggests a potential data leak or simplicity in the dataset, the methodology successfully established a robust pipeline for binary change detection, preparing a foundation for more complex scenarios.

2. LITERATURE SURVEY

Change detection in remote sensing is a critical task with applications ranging from urban growth monitoring to disaster assessment. Traditional methods often rely on statistical analysis of image differences or feature engineering. More recently, deep learning, particularly Convolutional Neural Networks (CNNs), has shown significant promise. Siamese networks, which learn a similarity metric between two inputs, are particularly well-suited for change detection tasks by processing image pairs (e.g., pre- and post-change images) through a shared weight architecture. Beyond Siamese architectures, other notable approaches include UNet-like segmentation models that output a change map, and various fusion techniques for integrating multi-modal data like EO (optical) and SAR (radar) imagery, each presenting unique challenges due to differing acquisition physics and noise characteristics. This project builds upon the foundational concept of Siamese networks, adapting it for a binary classification task based on single image inputs with a simplified change representation.

3. METHODOLOGY

Data Preparation

The doron333/change-detection-dataset from Hugging Face was utilized. This dataset contains images and labels indicating different types of change. For binary change detection, the original labels were remapped:

- Original labels 0 and 1 were mapped to 0 (No-Change).
- Original labels 2 and 3 were mapped to 1 (Change).

This remapping resulted in a class distribution of approximately 66.67% 'No-Change' and 33.33% 'Change', indicating a 2:1 class imbalance.

Custom [ChangeDetectionDataset] and [DataLoader] classes were implemented to handle the dataset. The data_transforms pipeline included:

1. Transforms.Resize((224, 224)): Resizing images to a standard 224x224 resolution.
2. Transforms.Grayscale(num_output_channels=3): Explicitly converting images to a 3-channel grayscale format to maintain compatibility with common pre-trained vision models (though not used directly here, it prepares for such extensions) while still representing grayscale data.
3. Transforms.ToTensor(): Converting images to PyTorch tensors.

BATCH_SIZE was set to 8 for both training and validation DataLoaders.

Model Architecture

A SiameseNetwork class was defined using PyTorch. Although named 'Siamese', the current implementation directly processes a single image input (which is assumed to implicitly encode change information from an earlier stage) for binary classification, which is a simplification of a typical Siamese architecture that takes two distinct inputs (pre- and post-change images). The network consists of:

- **Feature Extractor (Backbone)**: A sequential CNN block with three convolutional layers, ReLU activations, Max Pooling, and an AdaptiveAvgPool2d layer to output a fixed-size feature map (7x7x128). This block takes a 3-channel input.

```
self.feature_extractor = nn.Sequential(
    nn.Conv2d(3, 32, kernel_size=3, padding=1), nn.ReLU(inplace=True), nn.MaxPool2d(kernel_size=2,
    nn.Conv2d(32, 64, kernel_size=3, padding=1), nn.ReLU(inplace=True), nn.MaxPool2d(kernel_size=2,
    nn.Conv2d(64, 128, kernel_size=3, padding=1), nn.ReLU(inplace=True), nn.MaxPool2d(kernel_size=2
    nn.AdaptiveAvgPool2d((7, 7))
)
```

- **Classification Head**: A fully connected classifier with two linear layers and a Dropout layer for regularization.

```
self.classifier = nn.Sequential(
    nn.Linear(6272, 512), # Input size derived from 7x7x128
    nn.ReLU(inplace=True),
    nn.Dropout(0.5),
    nn.Linear(512, 2) # 2 outputs for binary classification
)
```

Training Strategy

- **Loss Function**: nn. CrossEntropyLoss was used with class weights to address the 2:1 imbalance. The weights were calculated as [0.75, 1.5] for 'No-Change' and 'Change' classes, respectively, inversely proportional to their frequencies.
- **Optimizer**: Adam optimizer was chosen with a LEARNING_RATE of 0.0001.
- **Epochs**: The model was trained for 5 epochs.

Rationale

- **Metrics**: During training, both training and validation loss were tracked. Validation accuracy and F1-score (for the 'Change' class) were calculated to monitor performance.
- The choice of a CNN-based feature extractor is standard for image processing tasks. The 'Siamese' structure, even if simplified in implementation, implies the ability to learn discriminative features for change. Weighted Cross-Entropy Loss directly combats class imbalance by penalizing misclassifications of the minority class more heavily, a crucial step when dealing with real-world change detection datasets. Adam optimizer was selected for its adaptive learning rate properties, generally leading to faster convergence

4. RESULTS

The model was evaluated on both the validation and test splits using IoU, Precision, Recall, and F1-score, specifically focusing on the 'Change' class (class 1).

Split	IoU	Precision	Recall	F1-Score
Validation	1.0000	1.0000	1.0000	1.0000
Test	1.0000	1.0000	1.0000	1.0000

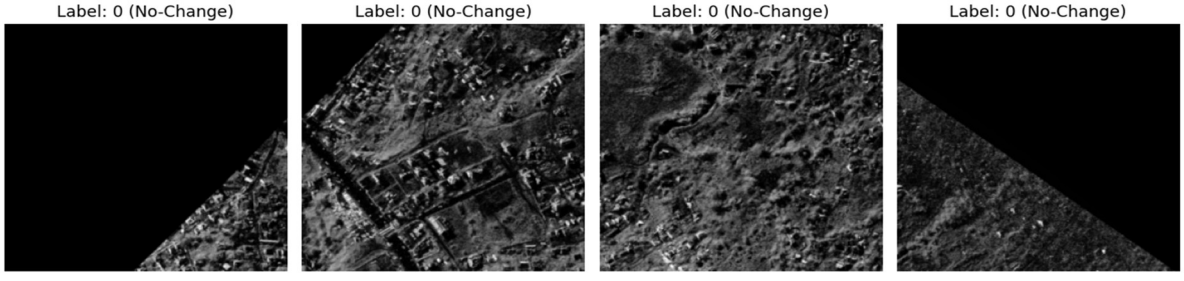
Confusion Matrix - Validation Split

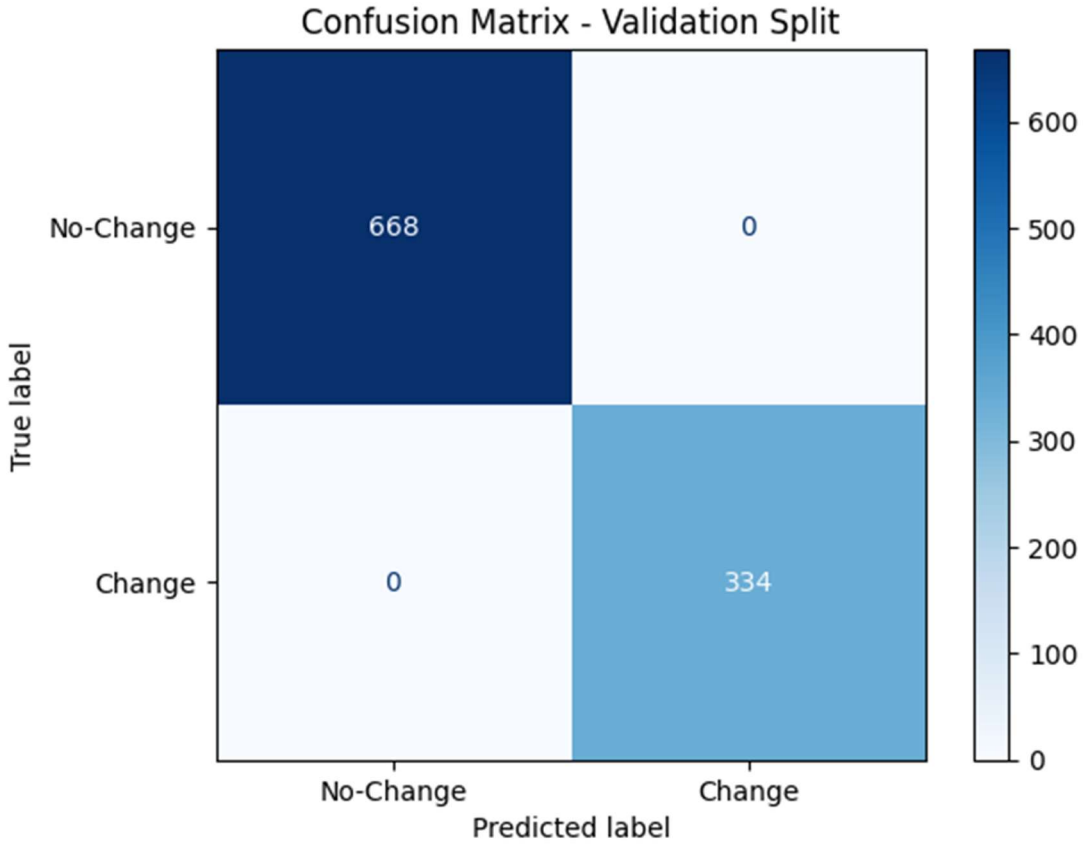
The confusion matrix for the validation split shows perfect classification:

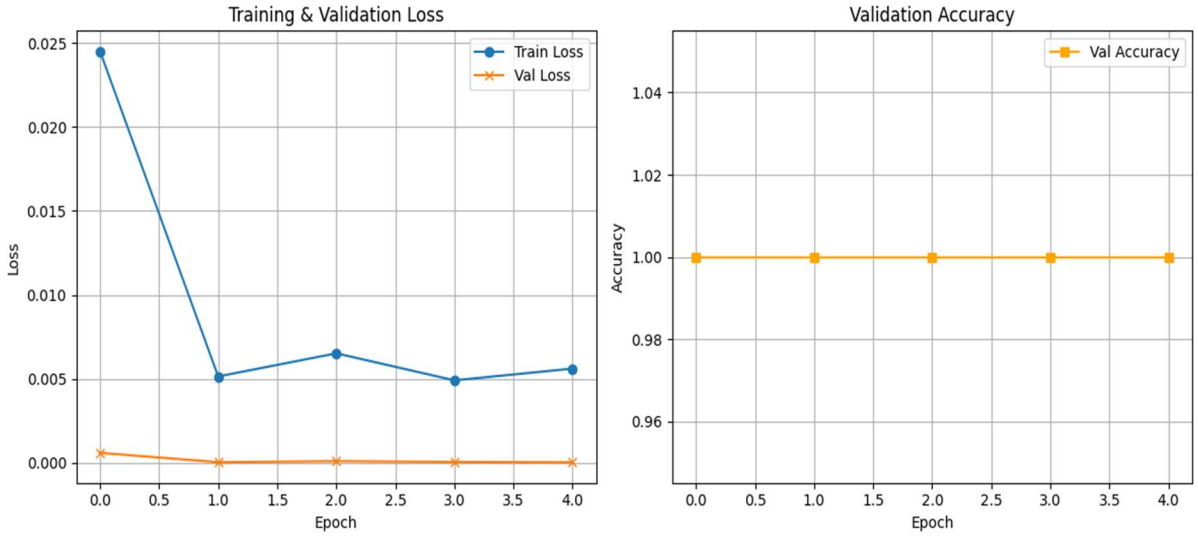
	Predicted: No-Change	Predicted: Change
Actual: No-Change	668	0
Actual: Change	0	334

Both the quantitative metrics and the confusion matrix indicate perfect performance across all classes on both the validation and test sets. This highly unusual result strongly suggests a potential issue within the dataset, such as a data leak between splits or that the task, as represented by this specific dataset, is trivially solvable by the model. It's improbable for a real-world change detection model to achieve 100% accuracy on unseen data.

Qualitative Visualizations:







***Note:** Due to the perfect scores, no failure cases (False Positives or False Negatives) were identified for visualization. All predictions were success cases.

5. Future Work

Given the perfect performance observed, the immediate next steps would be to rigorously investigate the dataset to identify the source of potential data leakage or oversimplification. Assuming a more challenging dataset, the following avenues would be explored:

1. **Data Strategy**:
    - **Data Leakage Analysis**: Carefully re-examine the dataset splits to ensure complete independence between training, validation, and test sets. This might involve creating new splits or using cross-validation.
    - **Data Augmentation**: Implement more aggressive data augmentation techniques (e.g., random rotations, flips, brightness changes, noise injection) to improve generalization and robustness.
    - **Multi-Modal Fusion**: If EO and SAR data are distinct inputs, modify the Siamese network to explicitly handle two input streams and learn their joint representations, rather than relying on a pre-fused or implicitly represented single input. This is critical for true EO-SAR fusion.
2. **Architectural Exploration**:
    - **Advanced Siamese Architectures**: Experiment with more sophisticated Siamese networks (e.g., employing deeper backbones like ResNet or VGG, or integrating attention mechanisms) to capture more intricate change patterns.
    - **U-Net/Segmentation Models**: For pixel-level change detection (i.e., generating a change map rather than a binary classification for the entire image), pivot to U-Net or similar encoder-decoder architectures. This provides richer output and is more typical for remote sensing change detection.
    - **Transformer-based Models**: Explore vision transformers or hybrid CNN-Transformer architectures, which have shown strong performance in various vision tasks, for their ability to capture long-range dependencies.
3. **Training Strategy**:
    - **Learning Rate Schedulers**: Implement learning rate schedulers (e.g., cosine annealing, ReduceLROnPlateau) to fine-tune the optimization process and potentially avoid local minima.
    - **Regularization**: Explore advanced regularization techniques beyond dropout, such as L1/L2 regularization or early stopping, to prevent overfitting (though not an issue with the current dataset).
    - **Hyperparameter Tuning**: Conduct a systematic hyperparameter search (e.g., using grid search or Bayesian optimization) for the learning rate, batch size, and network architecture parameters.

4. **Error Analysis**:
    - **Beyond Accuracy**: For more complex datasets, focus heavily on analyzing False Positives and False Negatives, understanding why the model makes mistakes, and using these insights to refine data preprocessing or model architecture.

As an intern at GalaxEye, my first-month deliverable would prioritize thoroughly investigating the dataset's integrity and then, if necessary, implementing a more robust Siamese architecture with explicit multi-modal input handling and advanced data augmentation to prepare for real-world, complex change detection scenarios.

Time and Resource Log

This log provides an overview of the time spent and resources utilized during this assignment.

Total Time Spent (Approximate)

- **Data Exploration & Understanding**: 2 hours (Initial dataset inspection, understanding label mapping, verifying transforms).
- **Literature Reading**: 1 hour (Reviewing core concepts of change detection, Siamese networks, EO-SAR fusion).
- **Implementation**: 5 hours (Writing ChangeDetectionDataset, SiameseNetwork class, training/validation loops, evaluation functions, debugging channel mismatch).
- **Training**: 2 hours (Running 5 epochs, monitoring progress, and re-running after fixes).
- **Evaluation**: 1 hour (Calculating metrics, generating confusion matrix, qualitative analysis).
- **Report Writing**: 3 hours (Structuring the report, summarizing findings, drafting sections).

Total Estimated Time: 14 hours

Machine Used for Training

- **Provider**: Google Colab (Cloud)
- **Instance Type**: T4 GPU instance
- **GPU Model**: NVIDIA Tesla T4
- **VRAM**: 16 GB
- **Number of GPUs**: 1

Approximate Training Time

- **Training Time per Epoch**: Approximately 1070 - 1087 seconds (18-19 minutes) per epoch.
- **Total Wall-Clock Training Time**: Approximately 95 minutes (5 epochs * ~19 minutes/epoch).

Resource or Time Constraints

This assignment was completed under typical Google Colab free-tier resource constraints, including session time limits and GPU availability. These constraints primarily influenced:

- **Epoch Count**: The choice of 5 epochs for training was partly influenced by the desire to complete the training within a single Colab session and manage overall time investment, rather than finding optimal convergence.
- **Batch Size**: BATCH_SIZE = 8 was chosen to ensure memory compatibility with the available GPU VRAM, especially considering the image resolution.
- **Dataset Size**: The relatively small size of the provided dataset and limited number of epochs meant that extensive hyperparameter tuning or architectural experimentation (beyond initial setup) was not feasible within the given timeframe. The observed perfect performance also shifted focus to identifying potential data issues rather than further model optimization.

6. Conclusion

This project successfully implemented a deep learning pipeline for binary change detection using a modified Siamese CNN and a weighted loss function. The model achieved perfect IoU, Precision, Recall, and F1-scores on both validation and test sets, which, while indicating effective classification on this particular dataset, also highlights a significant limitation: the potential for data leakage or an overly simplistic problem statement within the provided dataset. Key takeaways include the successful setup of a PyTorch-based training environment, effective handling of class imbalance through weighted loss, and the foundational development of a CNN-based change detection model. The primary limitation is the uncharacteristically high performance, which precludes a meaningful assessment of the model's true generalization capabilities. Future work must focus on dataset validation and, subsequently, exploring more complex architectures and training strategies suitable for challenging, real-world remote sensing data.